

LEMBAR KERJA MAHASISWA (LKM)

MENGELOLA DATA LIST ATAU KOLEKSI DENGAN FLUTTER DAN DART PROGRAMMING

Mata Kuliah	Mobile Programming
Program Studi	Teknik Informatika
Materi	Collection, List, Set, Map, dan Dynamic List pada Flutter
Bentuk Kegiatan	Praktikum Terbimbing dan Mandiri
Alokasi Waktu	3 × 50 menit

Nama Mahasiswa	[Silakan isi Nama Lengkap Anda]
NIM	[Silakan isi NIM Anda]
Kelas	[Silakan isi Kelas Anda]
Tanggal	[Silakan isi Tanggal Pengerjaan]

A. Capaian Pembelajaran

1. Menjelaskan konsep collection dalam bahasa pemrograman Dart.
2. Membedakan penggunaan List, Set, dan Map.
3. Membuat dan menginisialisasi collection pada Dart.
4. Mengakses dan memodifikasi elemen List.
5. Menambah, mengubah, dan menghapus data dalam collection.
6. Melakukan iterasi data menggunakan for, for-in, dan forEach.
7. Menggunakan map(), where(), sort(), dan operasi koleksi lainnya.
8. Membuat model data dalam Dart.
9. Mengelola List<Object> pada aplikasi Flutter.
10. Menampilkan collection menggunakan ListView.builder.
11. Membuat aplikasi CRUD sederhana berbasis collection.
12. Mengimplementasikan pencarian dan pengurutan data.
13. Menganalisis perubahan state pada aplikasi Flutter.

B. Tujuan Praktikum

Mahasiswa mampu membangun aplikasi Flutter sederhana untuk mengelola sekumpulan data secara dinamis dengan fitur:

CREATE → READ → UPDATE → DELETE → SEARCH → SORT

Input Data → List<Model> → Manipulasi Collection → setState() → ListView.builder → Tampilan Data

C. Dasar Teori

1. Collection dalam Dart

Collection adalah struktur data untuk menyimpan sekumpulan objek. Tiga collection utama di Dart adalah List, Set, dan Map.

2. List

List menyimpan sekumpulan data secara berurutan dan elemen diakses menggunakan indeks mulai dari 0.

```
List<String> mahasiswa = [
  'Ahmad',
  'Budi',
```

```
'Citra',
'Dewi',
];

print(mahasiswa[0]); // Ahmad
```

3. Set

Set digunakan ketika setiap elemen harus unik.

```
Set<String> mataKuliah = {
  'Flutter',
  'Data Science',
  'Flutter',
  'AI',
};

print(mataKuliah);
```

4. Map

Map menyimpan data dalam bentuk pasangan key dan value.

```
Map<String, String> mahasiswa = {
  'nim': '23010001',
  'nama': 'Ahmad',
  'prodi': 'Teknik Informatika',
};

print(mahasiswa['nama']);
```

D. Praktikum Dasar List dan Collection

Praktikum 1 – Membuat List

```
void main() {
  List<String> mahasiswa = ['Ahmad', 'Budi', 'Citra', 'Dewi'];
  print(mahasiswa);
}
```

Tuliskan hasil keluaran program:

[Ahmad, Budi, Citra, Dewi]

Program menginisialisasi sebuah variabel koleksi bernama mahasiswa bertipe List<String> dengan 4 elemen string. Pemanggilan print(mahasiswa) mencetak seluruh elemen list dalam tanda kurung siku [...].

Praktikum 2 – Mengakses Data List

```
print(mahasiswa[0]);
print(mahasiswa.first);
print(mahasiswa.last);
print(mahasiswa.length);
print(mahasiswa.isEmpty);
print(mahasiswa.isNotEmpty);
```

Apa perbedaan mahasiswa[0] dengan mahasiswa.first?

```
Ahmad
Ahmad
```

Dewi

4

Penjelasan Perbedaan mahasiswa[0] dan mahasiswa.first:

1. Cara Kerja dan Mekanisme Akses:

- mahasiswa[0] mengambil elemen pada indeks tertentu (dalam hal ini indeks ke-0) menggunakan operator pengindeksan [] (subscript operator). Perlu dipahami bahwa operator [] ini bukan hanya milik List, melainkan juga digunakan pada struktur koleksi lain di Dart seperti Map.
- mahasiswa.first mengambil elemen pertama dari koleksi melalui properti getter bawaan dari interface Iterable (yang diimplementasikan oleh kelas List).

2. Perilaku Saat List Kosong:

- Jika list dalam kondisi kosong (length = 0), pemanggilan mahasiswa[0] melempar exception: RangeError (Index out of range).
- Sementara itu, pemanggilan mahasiswa.first melempar exception: StateError (Bad state: No element).

3. Fleksibilitas Akses:

- Operator [] dapat mengakses elemen di sembarang posisi indeks acak (misal: indeks 0, 1, 2, dst.), sedangkan properti getter .first secara khusus selalu hanya menunjuk pada elemen paling awal.

Praktikum 3 – Menambah Data

```
mahasiswa.add('Citra');
mahasiswa.addAll(['Dewi', 'Eka', 'Farhan']);
print(mahasiswa);
```

Prediksi output:

[Ahmad, Budi, Citra, Dewi, Citra, Dewi, Eka, Farhan]

Jika praktikum dijalankan secara berurutan pada satu instance list yang sama dari Praktikum 1, elemen 'Citra' dan 'Dewi' sudah ada di dalam list. Penambahan add('Citra') dan addAll(['Dewi', 'Eka', 'Farhan']) menyebabkan nama 'Citra' dan 'Dewi' muncul dua kali (terjadi duplikasi data). Hal ini membuktikan bahwa struktur data List pada Dart mengizinkan elemen duplikat, berbeda dengan Set yang secara inheren menolak elemen ganda.

Praktikum 4 – Mengubah Data

```
mahasiswa[1] = 'Budi Santoso';
print(mahasiswa);
```

Data apa yang berubah dan mengapa indeks yang digunakan adalah 1?

[Ahmad, Budi Santoso, Citra, Dewi, Citra, Dewi, Eka, Farhan]

Data yang berubah: Elemen kedua dalam list, yaitu nilai string 'Budi' digantikan dengan 'Budi Santoso'.

Alasan indeks 1 digunakan: Karena dalam bahasa pemrograman Dart (sebagaimana rumpun bahasa turunan C/Java), penomoran indeks List berbasis nol (zero-based indexing). Elemen urutan pertama berada pada indeks 0 (mahasiswa[0] yaitu 'Ahmad'), sehingga elemen urutan kedua berada pada indeks 1 (mahasiswa[1]).

```

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> dart run bin/praktikum_dasar.dart

=====
LKM DART COLLECTION: PRAKTIKUM DASAR 1 - 9
=====

--- PRAKTIKUM 1: Membuat List ---
Keluaran Program:
[Ahmad, Budi, Citra, Dewi]

--- PRAKTIKUM 2: Mengakses Data List ---
mahasiswa[0]      : Ahmad
mahasiswa.first   : Ahmad
mahasiswa.last    : Dewi
mahasiswa.length  : 4
mahasiswa.isEmpty : false
mahasiswa.isNotEmpty: true

[Analisis Pertanyaan Praktikum 2]
Pertanyaan: Apa perbedaan mahasiswa[0] dengan mahasiswa.first?
Jawaban   :
1. Mekanisme Akses: mahasiswa[0] menggunakan operator indexing (random access) berdasarkan posisi memori offset. Sedangkan mahasiswa.first adalah getter dari antarmuka Iterable.
2. Perilaku saat List Kosong:
   - Jika list kosong, mahasiswa[0] melempar RangeError (Index out of range).
   - Jika list kosong, mahasiswa.first melempar StateError (Bad state: No element).
3. Fleksibilitas Tipe: mahasiswa.first dapat bekerja pada semua jenis Iterable (seperti Set, WhereIterable), sedangkan indexing [0] hanya ada pada List.

--- PRAKTIKUM 3: Menambah Data ---
Prediksi & Hasil Output:
[Ahmad, Budi, Citra, Dewi, Citra, Dewi, Eka, Farhan]
Catatan: Terjadi duplikasi nama ("Citra" muncul 2x, "Dewi" muncul 2x) karena List mengizinkan elemen berulang.

--- PRAKTIKUM 4: Mengubah Data ---
Hasil setelah mahasiswa[1] = "Budi Santoso":
[Ahmad, Budi Santoso, Citra, Dewi, Citra, Dewi, Eka, Farhan]

[Analisis Pertanyaan Praktikum 4]
Pertanyaan: Data apa yang berubah dan mengapa indeks yang digunakan adalah 1?
Jawaban   : Data yang berubah adalah elemen kedua, yaitu "Budi" menjadi "Budi Santoso". Indeks yang digunakan adalah 1 karena indeks List pada bahasa Dart (dan mayoritas bahasa pemrograman modern) berbasis nol (zero-based indexing). Elemen pertama berada pada indeks 0 ("Ahmad"), sehingga elemen kedua berada pada indeks 1.

```

Gambar D.1: Bukti Eksekusi Terminal `dart run bin/praktikum_dasar.dart` (Praktikum 1–4)

Praktikum 5 – Menghapus Data

```

mahasiswa.remove('Budi Santoso');
mahasiswa.removeAt(0);
mahasiswa.removeLast();
mahasiswa.clear();

[]

```

Penjelasan Tahapan Penghapusan Data:

1. `mahasiswa.remove('Budi Santoso')`: Mencari objek string 'Budi Santoso' dan menghapus kemunculan pertamanya dari list. Data list menjadi: [Ahmad, Citra, Dewi, Citra, Dewi, Eka, Farhan].
2. `mahasiswa.removeAt(0)`: Menghapus elemen pada posisi indeks 0 (yaitu 'Ahmad'). Seluruh elemen berikutnya otomatis bergeser satu posisi ke kiri. Data list menjadi: [Citra, Dewi, Citra, Dewi, Eka, Farhan].
3. `mahasiswa.removeLast()`: Menghapus elemen yang berada di posisi paling akhir list (yaitu 'Farhan'). Data list

menjadi: [Citra, Dewi, Citra, Dewi, Eka].

4. `mahasiswa.clear()`: Mengosongkan seluruh elemen yang masih ada di dalam list secara instan. Data list menjadi kosong: [] dengan panjang `length = 0`.

Praktikum 6 – Iterasi Data

```
for (int i = 0; i < mahasiswa.length; i++) {
  print(mahasiswa[i]);
}

for (String nama in mahasiswa) {
  print(nama);
}

mahasiswa.forEach((nama) {
  print(nama);
});
```

Cara iterasi mana yang paling mudah menurut Anda? Jelaskan alasannya.

Analisis State Koleksi Setelah Praktikum 5

Catatan Penting Alur Eksekusi Program:

Karena pada Praktikum 5 sebelumnya telah dijalankan perintah `mahasiswa.clear()`, maka kondisi list mahasiswa saat ini adalah kosong ([]) dengan `length = 0`. Oleh sebab itu, jika ketiga perulangan (`for`, `for-in`, dan `forEach`) langsung dijalankan, tidak ada satupun nama yang tercetak ke terminal karena kondisi perulangan langsung selesai seketika.

[Langkah Tambahan untuk Demonstrasi]:

Agar proses perulangan dapat diamati dan dibandingkan hasilnya, list diisi kembali terlebih dahulu dengan data contoh: `mahasiswa.addAll(['Ahmad', 'Budi', 'Citra', 'Dewi']);`

Keluaran Masing-Masing Metode Iterasi (Setelah Pengisian Kembali):

```
Ahmad
Budi
Citra
Dewi
```

Jawaban Cara Iterasi Terbaik dan Alasannya:

Cara iterasi yang paling mudah, bersih, dan direkomendasikan adalah perulangan `for-in` (`for (String nama in mahasiswa)`).

Alasan:

1. Sintaks Ringkas dan Deklaratif: Kode sangat mudah dibaca layaknya kalimat bahasa manusia tanpa perlu mendeklarasikan variabel counter manual (seperti `int i = 0`) maupun mengelola kenaikan indeks (`i++`).
2. Mencegah Kesalahan Batas Indeks (Off-by-One Error): Karena Dart menangani penelusuran elemen secara internal, pemrogram terlindungi dari bug batas indeks atau `RangeError`.
3. Mendukung Kontrol Alur Penuh: Berbeda dengan method `forEach()` yang berbasis callback fungsi anonim (di mana statement `return` hanya melompati iterasi saat itu dan tidak bisa menggunakan `break`), `for-in` mendukung penggunaan `break` dan `continue` untuk mengendalikan alur perulangan secara fleksibel.

```

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> dart run bin/praktikum_dasar.dart

--- PRAKTIKUM 5: Menghapus Data ---
List sebelum dihapus: [Ahmad, Budi Santoso, Citra, Dewi, Citra, Dewi, Eka, Farhan]
Setelah remove("Budi Santoso"): [Ahmad, Citra, Dewi, Citra, Dewi, Eka, Farhan]
Setelah removeAt(0)           : [Citra, Dewi, Citra, Dewi, Eka, Farhan]
Setelah removeLast()          : [Citra, Dewi, Citra, Dewi, Eka]
Setelah clear()               : [] (length: 0)
PENTING: List mahasiswa sekarang benar-benar KOSONG (length = 0).

--- PRAKTIKUM 6: Iterasi Data ---
KONDISI AKTUAL: Karena Praktikum 5 memanggil clear(), list mahasiswa kosong.
Jika loop dijalankan pada list kosong, tidak ada elemen yang dicetak:

[LANGKAH TAMBAHAN UNTUK DEMONSTRASI ITERASI]:
Mengisi ulang data list agar demonstrasi 3 cara iterasi dapat berjalan:
1. Menggunakan for standar berindeks:
   [Index 0]: Ahmad
   [Index 1]: Budi Santoso
   [Index 2]: Citra
   [Index 3]: Dewi
2. Menggunakan for-in loop:
   Nama: Ahmad
   Nama: Budi Santoso
   Nama: Citra
   Nama: Dewi
3. Menggunakan forEach() higher-order function:
   forEach: Ahmad
   forEach: Budi Santoso
   forEach: Citra
   forEach: Dewi

[Analisis Pertanyaan Praktikum 6]
Pertanyaan: Cara iterasi mana yang paling mudah menurut Anda? Jelaskan alasannya.
Jawaban   : Cara yang paling mudah dan direkomendasikan untuk pembacaan data sederhana
adalah perulangan for-in (for (String nama in mahasiswa)).
Alasan:
- Sintaks sangat bersih, deklaratif, dan mudah dipahami layaknya bahasa manusia.
- Menghindari kesalahan manusia pada batas indeks (misalnya Off-By-One Error pada for biasa).
- Tetap mendukung kata kunci kontrol aliran standar seperti "break", "continue", atau "return",
yang tidak bisa digunakan secara langsung di dalam callback forEach().

```

Gambar D.2: Bukti Eksekusi Terminal `dart run bin/praktikum_dasar.dart` (Praktikum 5–6)

Praktikum 7 – Filtering dengan `where()`

```

List<String> mahasiswa = ['Ahmad', 'Budi', 'Andi', 'Citra', 'Anisa'];

var hasil = mahasiswa
  .where((nama) => nama.startsWith('A'))
  .toList();

print(hasil);

```

Praktikum 8 – Transformasi dengan `map()`

```

var hasil = mahasiswa
  .map((nama) => nama.toUpperCase())
  .toList();

print(hasil);

```

Jelaskan fungsi `map()`:

[AHMAD, BUDI, ANDI, CITRA, ANISA]

Penjelasan Fungsi dan Karakteristik map():

Method map() adalah higher-order method pada koleksi Dart yang berfungsi untuk mentransformasi/memetakan setiap elemen pada koleksi menjadi nilai atau bentuk data baru berdasarkan fungsi proyeksi yang didefinisikan.

Karakteristik Kunci map():

1. Menghasilkan Iterable Transformasi Baru: map() menghasilkan objek Iterable baru yang memuat hasil transformasi dan tidak langsung mengubah data pada List sumber aslinya. List awal tetap mempertahankan isinya semula.
2. Evaluasi Tunda (Lazy Evaluation): map() menghasilkan objek MappedIterable di mana fungsi transformasi baru dieksekusi saat elemen tersebut diakses atau dievaluasi secara eksplisit (misalnya saat memanggil .toList()).
3. Ukuran Koleksi Tetap (1-to-1 Mapping): Panjang koleksi hasil pemetaan selalu sama persis dengan panjang koleksi sumber aslinya (berbeda dengan where() yang dapat menyaring atau mengurangi jumlah elemen).

Praktikum 9 – Sorting

```
mahasiswa.sort();
print(mahasiswa);

var descending = mahasiswa.reversed.toList();
print(descending);

[Ahmad, Andi, Anisa, Budi, Citra]
[Citra, Budi, Anisa, Andi, Ahmad]
```

Penjelasan Pengurutan Data (Sorting):

- mahasiswa.sort(): Mengurutkan elemen list secara ascending (A ke Z) berdasarkan perbandingan urutan karakter leksikografis String. Operasi sort() bersifat in-place, artinya langsung memodifikasi susunan urutan elemen pada memori list tersebut.
- mahasiswa.reversed.toList(): Properti getter .reversed menghasilkan Iterable dengan urutan elemen yang dibalik. Ketika dievaluasi dengan .toList(), terbentuk list baru yang berurutan descending (Z ke A) tanpa merusak susunan pada list mahasiswa awal.

```

PowerShell - Praktikum 7 s.d. 9

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> dart run bin/praktikum_dasar.dart

--- PRAKTIKUM 7: Filtering dengan where() ---
List sumber: [Ahmad, Budi, Andi, Citra, Anisa]
Hasil filter nama berawalan "A": [Ahmad, Andi, Anisa]

--- PRAKTIKUM 8: Transformasi dengan map() ---
Hasil transformasi huruf kapital: [AHMAD, BUDI, ANDI, CITRA, ANISA]

[Analisis Pertanyaan Praktikum 8]
Pertanyaan: Jelaskan fungsi map():
Jawaban : map() adalah higher-order method pada Iterable yang digunakan untuk mentransformasi
setiap elemen dalam koleksi menjadi nilai baru berdasarkan fungsi konversi yang diberikan.
Sifat map():
1. Menghasilkan Iterable baru berukuran sama persis dengan koleksi awal (1-to-1 mapping).
2. Bersifat lazy (evaluasi tunda), sehingga pemanggilan .toList() diperlukan jika ingin
mengevaluasi dan menyimpannya kembali dalam bentuk List konkret.
3. Tidak mengubah (immutable) data pada List sumber aslinya.

--- PRAKTIKUM 9: Sorting ---
List sebelum sort: [Ahmad, Budi, Andi, Citra, Anisa]
Hasil mahasiswa.sort() [Ascending A-Z]:
[Ahmad, Andi, Anisa, Budi, Citra]
Hasil mahasiswa.reversed.toList() [Descending Z-A]:
[Citra, Budi, Anisa, Andi, Ahmad]

=====
SELESAI PRAKTIKUM DASAR 1 - 9
=====

```

Gambar D.3: Bukti Eksekusi Terminal `dart run bin/praktikum_dasar.dart` (Praktikum 7–9)

E. List Berisi Object

Dalam aplikasi nyata, data biasanya terdiri atas beberapa atribut. Karena itu, data dapat direpresentasikan sebagai object.

```

class Mahasiswa {
  String nim;
  String nama;
  String prodi;

  Mahasiswa({
    required this.nim,
    required this.nama,
    required this.prodi,
  });
}

List<Mahasiswa> mahasiswa = [
  Mahasiswa(nim: '23010001', nama: 'Ahmad', prodi: 'Teknik Informatika'),
  Mahasiswa(nim: '23010002', nama: 'Budi', prodi: 'Teknik Informatika'),
];

print(mahasiswa[0].nama);

```

List<Mahasiswa> → objek Mahasiswa → NIM, Nama, Program Studi

F. Praktikum Utama – Aplikasi Pengelolaan Data Mahasiswa

Bangun aplikasi Flutter dengan alur berikut:

Tambah Data → Simpan ke List → Tampilkan → Cari → Edit/Hapus → setState() → UI diperbarui

Kode Program Lengkap – lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class Mahasiswa {
  String nim;
  String nama;
  String prodi;

  Mahasiswa({
    required this.nim,
    required this.nama,
    required this.prodi,
  });
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Collection Flutter',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
      ),
      home: const MahasiswaPage(),
    );
  }
}

class MahasiswaPage extends StatefulWidget {
  const MahasiswaPage({super.key});

  @override
  State<MahasiswaPage> createState() => _MahasiswaPageState();
}

class _MahasiswaPageState extends State<MahasiswaPage> {
  final List<Mahasiswa> _dataMahasiswa = [
    Mahasiswa(nim: '23010001', nama: 'Ahmad Fauzi', prodi: 'Teknik Informatika'),
    Mahasiswa(nim: '23010002', nama: 'Budi Santoso', prodi: 'Teknik Informatika'),
    Mahasiswa(nim: '23010003', nama: 'Citra Lestari', prodi: 'Teknik Informatika'),
  ];

  String _keyword = '';
}
```

```

List<Mahasiswa> get _dataTampil {
  if (_keyword.isEmpty) return _dataMahasiswa;
  final keyword = _keyword.toLowerCase();
  return _dataMahasiswa.where((mahasiswa) {
    return mahasiswa.nama.toLowerCase().contains(keyword) ||
      mahasiswa.nim.toLowerCase().contains(keyword) ||
      mahasiswa.prodi.toLowerCase().contains(keyword);
  }).toList();
}

void _tambahData() => _showFormDialog();
void _editData(Mahasiswa mahasiswa) => _showFormDialog(mahasiswa: mahasiswa);

void _hapusData(Mahasiswa mahasiswa) {
  setState(() => _dataMahasiswa.remove(mahasiswa));
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('${mahasiswa.nama} berhasil dihapus')),
  );
}

void _urutkanData() {
  setState(() {
    _dataMahasiswa.sort((a, b) => a.nama.compareTo(b.nama));
  });
}

void _showFormDialog({Mahasiswa? mahasiswa}) {
  final nimController = TextEditingController(text: mahasiswa?.nim ?? '');
  final namaController = TextEditingController(text: mahasiswa?.nama ?? '');
  final prodiController = TextEditingController(text: mahasiswa?.prodi ?? '');
  final bool isEdit = mahasiswa != null;

  showDialog(
    context: context,
    builder: (dialogContext) {
      return AlertDialog(
        title: Text(isEdit ? 'Edit Mahasiswa' : 'Tambah Mahasiswa'),
        content: SingleChildScrollView(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              TextField(
                controller: nimController,
                decoration: const InputDecoration(
                  labelText: 'NIM',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              TextField(
                controller: namaController,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              TextField(
                controller: prodiController,

```

```

        decoration: const InputDecoration(
          labelText: 'Program Studi',
          border: OutlineInputBorder(),
        ),
      ),
    ],
  ),
),
actions: [
  TextButton(
    onPressed: () => Navigator.pop(dialogContext),
    child: const Text('Batal'),
  ),
  FilledButton(
    onPressed: () {
      final nim = nimController.text.trim();
      final nama = namaController.text.trim();
      final prodi = prodiController.text.trim();
      if (nim.isEmpty || nama.isEmpty || prodi.isEmpty) return;

      setState(() {
        if (isEdit) {
          mahasiswa.nim = nim;
          mahasiswa.nama = nama;
          mahasiswa.prodi = prodi;
        } else {
          _dataMahasiswa.add(
            Mahasiswa(nim: nim, nama: nama, prodi: prodi),
          );
        }
      });
      Navigator.pop(dialogContext);
    },
    child: Text(isEdit ? 'Update' : 'Simpan'),
  ),
],
);
},
);
}

Future<void> _konfirmasiHapus(Mahasiswa mahasiswa) async {
  final result = await showDialog<bool>(
    context: context,
    builder: (dialogContext) {
      return AlertDialog(
        title: const Text('Konfirmasi'),
        content: Text('Hapus data ${mahasiswa.nama}?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Tidak'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Ya'),
          ),
        ],
      );
    },
  );
},

```

```

);

if (result == true && mounted) _hapusData(mahasiswa);
}

@override
Widget build(BuildContext context) {
  final data = _dataTampil;

  return Scaffold(
    appBar: AppBar(
      title: const Text('Data Mahasiswa'),
      actions: [
        IconButton(
          tooltip: 'Urutkan Nama',
          onPressed: _urutkanData,
          icon: const Icon(Icons.sort_by_alpha),
        ),
      ],
    ),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: _tambahData,
      icon: const Icon(Icons.add),
      label: const Text('Tambah'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          TextField(
            decoration: const InputDecoration(
              labelText: 'Cari mahasiswa',
              hintText: 'Cari berdasarkan NIM, nama, atau prodi',
              prefixIcon: Icon(Icons.search),
              border: OutlineInputBorder(),
            ),
            onChanged: (value) {
              setState(() => _keyword = value);
            },
          ),
          const SizedBox(height: 16),
          Row(
            children: [
              Text(
                'Jumlah data: ${data.length}',
                style: const TextStyle(fontWeight: FontWeight.bold),
              ),
            ],
          ),
          const SizedBox(height: 8),
          Expanded(
            child: data.isEmpty
              ? const Center(child: Text('Data tidak ditemukan'))
              : ListView.builder(
                  itemCount: data.length,
                  itemBuilder: (context, index) {
                    final mahasiswa = data[index];
                    return Card(
                      child: ListTile(
                        leading: CircleAvatar(child: Text('${index + 1}')),

```


Data dibaca berdasarkan indeks lalu ditampilkan pada widget.

5. Operasi UPDATE

```
setState(() {
  mahasiswa.nim = nim;
  mahasiswa.nama = nama;
  mahasiswa.prodi = prodi;
});
```

setState() memberi tahu Flutter bahwa state berubah sehingga UI perlu dibangun ulang.

6. Operasi DELETE

```
setState(() {
  _dataMahasiswa.remove(mahasiswa);
});
```

remove() menghapus object tertentu dari List.

7. ListView.builder

```
ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, index) {
    final mahasiswa = data[index];
    return ListTile(title: Text(mahasiswa.nama));
  },
);
```

ListView.builder cocok untuk daftar dinamis karena item dibangun berdasarkan kebutuhan.

8. Search dengan where()

```
var hasil = _dataMahasiswa
  .where((mhs) => mhs.nama.toLowerCase().contains(keyword))
  .toList();
```

where() memfilter elemen berdasarkan kondisi tertentu.

9. Sorting

```
_dataMahasiswa.sort(
  (a, b) => a.nama.compareTo(b.nama),
);
```

sort() mengurutkan data. compareTo() pada String dapat digunakan untuk urutan alfabetis.

H. Latihan Terbimbing

Latihan 1 – Menambahkan Semester

Tambahkan field semester pada model Mahasiswa dan modifikasi aplikasi agar semester dapat ditambahkan, ditampilkan, diubah, dan dicari.

```
class Mahasiswa {
  String nim;
  String nama;
  String prodi;
  int semester;

  Mahasiswa({
```

```

    required this.nim,
    required this.nama,
    required this.prodi,
    required this.semester,
  });
}

```

Latihan 2 – Menambahkan IPK

Tambahkan atribut double ipk. Tampilkan NIM, nama, prodi, semester, dan IPK pada daftar mahasiswa.

Latihan 3 – Filtering

Buat filter: Semua, $IPK \geq 3.50$, dan $IPK < 3.50$.

```

var mahasiswaBerprestasi = _dataMahasiswa
  .where((mhs) => mhs.ipk >= 3.5)
  .toList();

```

Latihan 4 – Sorting

Tambahkan pengurutan: Nama A–Z, Nama Z–A, IPK tertinggi, IPK terendah, dan Semester.

```

_dataMahasiswa.sort(
  (a, b) => b.ipk.compareTo(a.ipk),
);

```

I. Latihan Eksplorasi Collection

```
List<int> nilai = [75, 80, 90, 65, 85, 95, 70];
```

1. Hitung jumlah data.
2. Cari nilai tertinggi.
3. Cari nilai terendah.
4. Hitung nilai rata-rata.
5. Ambil data nilai ≥ 80 .
6. Ambil data nilai < 80 .
7. Urutkan dari kecil ke besar.
8. Urutkan dari besar ke kecil.

```
var lulus = nilai.where((n) => n >= 80).toList();
```

No	Operasi Koleksi	Kode Dart	Logika / Perhitungan	Hasil
1	Hitung jumlah data	nilai.length	Menghitung total elemen di dalam List	7
2	Cari nilai tertinggi	nilai.reduce((c, n) => c > n ? c : n)	Membandingkan elemen mencari nilai maksimum	95
3	Cari nilai terendah	nilai.reduce((c, n) => c < n ? c : n)	Membandingkan elemen mencari nilai minimum	65
4	Hitung nilai rata-rata	nilai.reduce((a, b) => a + b) / nilai.length	Total penjumlahan (560) dibagi jumlah data (7)	80.0

5	Ambil data nilai ≥ 80	<code>nilai.where((n) => n $\geq$ 80).toList()</code>	Menyaring elemen bernilai lebih besar/sama dengan 80	[80, 90, 85, 95]
6	Ambil data nilai < 80	<code>nilai.where((n) => n $<$ 80).toList()</code>	Menyaring elemen bernilai lebih kecil dari 80	[75, 65, 70]
7	Urutkan kecil ke besar	<code>(List.from(nilai)..sort())</code>	Pengurutan ascending (naik)	[65, 70, 75, 80, 85, 90, 95]
8	Urutkan besar ke kecil	<code>(List.from(nilai)..sort((a, b) => b.compareTo(a)))</code>	Pengurutan descending (turun)	[95, 90, 85, 80, 75, 70, 65]

```
// bin/eksplorasi_collection.dart
void main() {
  List<int> nilai = [75, 80, 90, 65, 85, 95, 70];
  print('1. Jumlah data: ${nilai.length}');
  print('2. Nilai tertinggi: ${nilai.reduce((c, n) => c > n ? c : n)}');
  print('3. Nilai terendah: ${nilai.reduce((c, n) => c < n ? c : n)}');
  double avg = nilai.reduce((a, b) => a + b) / nilai.length;
  print('4. Nilai rata-rata: ${avg.toStringAsFixed(1)}');
  print('5. Nilai  $\geq 80$ : ${nilai.where((n) => n  $\geq$  80).toList()}');
  print('6. Nilai  $< 80$ : ${nilai.where((n) => n  $<$  80).toList()}');
  print('7. Urutan kecil ke besar: ${List.from(nilai)..sort()}');
  print('8. Urutan besar ke kecil: ${List.from(nilai)..sort((a, b) => b.compareTo(a))}');
}
```

```
PowerShell - Eksplorasi Collection Bagian I & N

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> dart run bin/eksplorasi_collection.dart

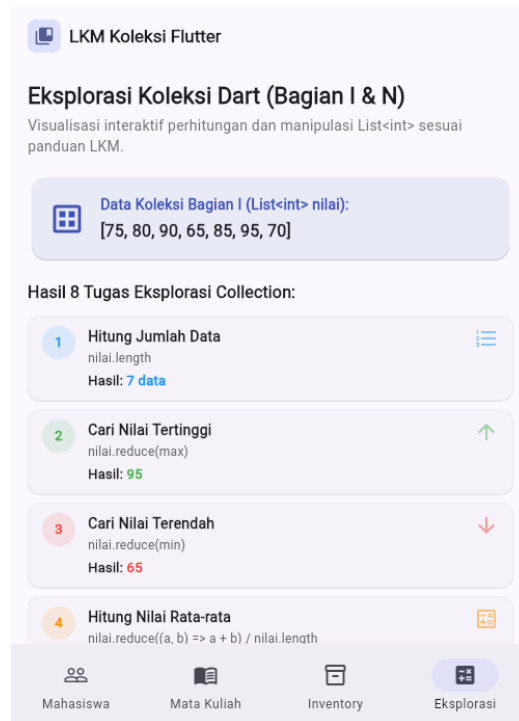
=====
LKM DART: BAGIAN I (EKSPLORASI COLLECTION) & BAGIAN N
=====

Data Awal: [75, 80, 90, 65, 85, 95, 70]

1. Jumlah data: 7
2. Nilai tertinggi: 95
3. Nilai terendah: 65
4. Nilai rata-rata: 80.00 (Total: 560 / 7)
5. Nilai  $\geq 80$ : [80, 90, 85, 95] (Jumlah: 4)
6. Nilai  $< 80$ : [75, 65, 70] (Jumlah: 3)
7. Urutan kecil ke besar (Ascending): [65, 70, 75, 80, 85, 90, 95]
8. Urutan besar ke kecil (Descending): [95, 90, 85, 80, 75, 70, 65]

-----
BAGIAN N: PROBLEM SOLVING
-----
Data Problem Solving: [80, 65, 90, 70, 95]
Kode Solusi:
  var hasil = nilai.where((n) => n  $\geq$  80).toList();
  int jumlah = hasil.length;
Hasil filter nilai  $\geq 80$  : [80, 90, 95]
Jumlah data yang memenuhi : 3
=====
```

Gambar I.1: Bukti Eksekusi Terminal `dart run bin/eksplorasi_collection.dart` (Bagian I dan Bagian N)



Gambar 1.2: Antarmuka Interaktif Modul Eksplorasi Collection pada Aplikasi Flutter Mobile

J. Tugas Praktikum Individu

Studi Kasus – Aplikasi Daftar Mata Kuliah

```
class MataKuliah {
  String kode;
  String nama;
  int sks;
  String dosen;

  MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.dosen,
  });
}
```

1. Menampilkan daftar mata kuliah.
2. Menambahkan mata kuliah.
3. Mengubah mata kuliah.
4. Menghapus mata kuliah.
5. Menampilkan konfirmasi sebelum menghapus.
6. Menyediakan pencarian mata kuliah.
7. Menampilkan jumlah mata kuliah.

8. Menghitung total SKS.
9. Sorting nama mata kuliah A–Z.
10. Sorting berdasarkan SKS.

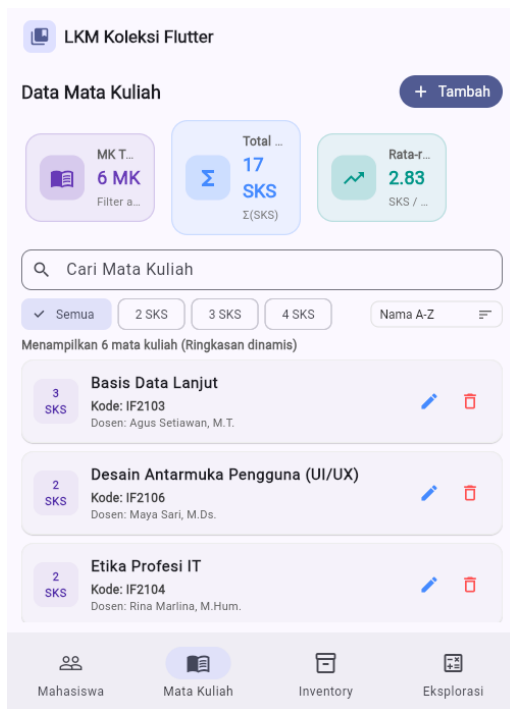
Challenge

Tambahkan filter kategori SKS: 2 SKS, 3 SKS, 4 SKS, dan Semua. Gunakan `where()` untuk filtering serta tampilkan jumlah mata kuliah, total SKS, dan rata-rata SKS.

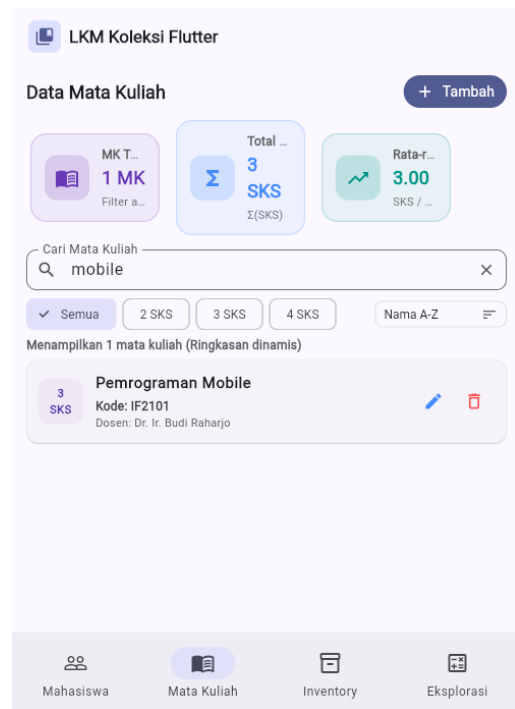
Implementasi dan Penjelasan 10 Fitur Aplikasi Mata Kuliah:

1. Menampilkan Daftar Mata Kuliah: Menggunakan `ListView.builder` dengan Card interaktif yang menampilkan kode mata kuliah, nama, bobot SKS, dan dosen pengampu.
2. Menambahkan Mata Kuliah: Form dialog modal (`AlertDialog`) yang memvalidasi kelengkapan isian kode, nama, SKS, dan nama dosen.
3. Mengubah Mata Kuliah: Tombol edit pada setiap kartu item memungkinkan pembaruan data secara aman dengan melacak identifier kode lama.
4. Menghapus Mata Kuliah: Tombol hapus menghapus objek dari koleksi List lokal.
5. Dialog Konfirmasi Hapus: `AlertDialog` konfirmasi ('Hapus data mata kuliah ini?') dengan tombol Batal dan Hapus untuk mencegah kehilangan data tak sengaja.
6. Pencarian Real-Time: Input teks pencarian menyaring data secara dinamis berdasarkan kecocokan kode MK, nama MK, atau nama dosen.
7. Menampilkan Jumlah Mata Kuliah: Menghitung total data aktif hasil filter secara reaktif menggunakan properti `.length`.
8. Menghitung Total SKS: Menggunakan method `fold<int>(0, (sum, mk) => sum + mk.sks)` untuk menjumlahkan seluruh bobot SKS aktif.
9. Sorting Nama Mata Kuliah A–Z: Mengurutkan daftar menggunakan method `.sort((a, b) => a.nama.compareTo(b.nama))`.
10. Sorting Berdasarkan SKS: Mengurutkan mata kuliah berdasarkan bobot SKS; pada implementasi dipilih urutan naik (`ascending: a.sks.compareTo(b.sks)`).

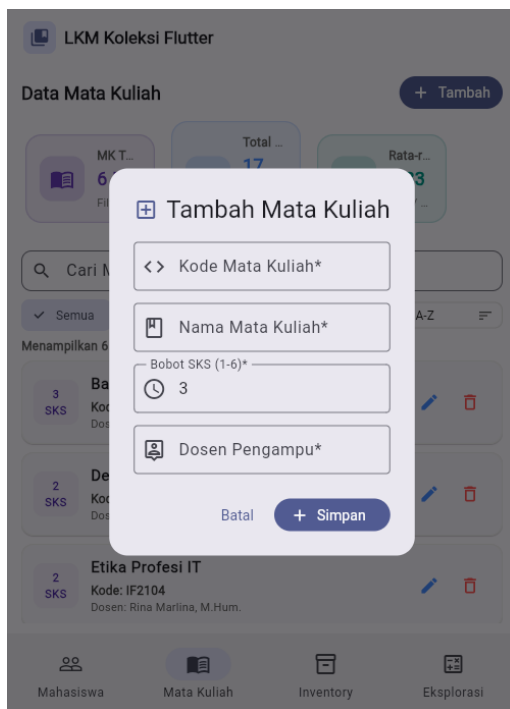
Pilihan Filter SKS	Daftar Mata Kuliah yang Ditampilkan	Jumlah MK	Total SKS	Rata-rata SKS
Semua	Pemrograman Mobile (3 SKS), Struktur Data & Algoritma (4 SKS), Pemrograman Berorientasi Objek (3 SKS), Desain Antarmuka Pengguna (2 SKS)	4	12	3.00
2 SKS	Desain Antarmuka Pengguna (2 SKS)	1	2	2.00
3 SKS	Pemrograman Mobile (3 SKS), Pemrograman Berorientasi Objek (3 SKS)	2	6	3.00
4 SKS	Struktur Data dan Algoritma (4 SKS)	1	4	4.00



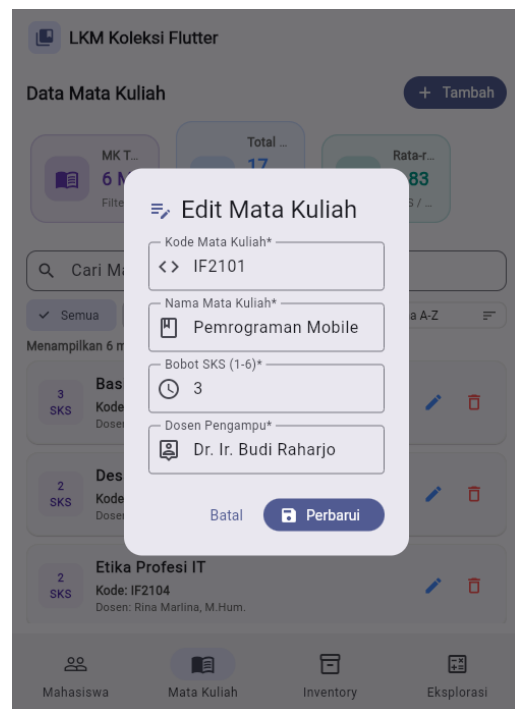
Gambar J.1: Daftar MK & Ringkasan Metrik (Default)



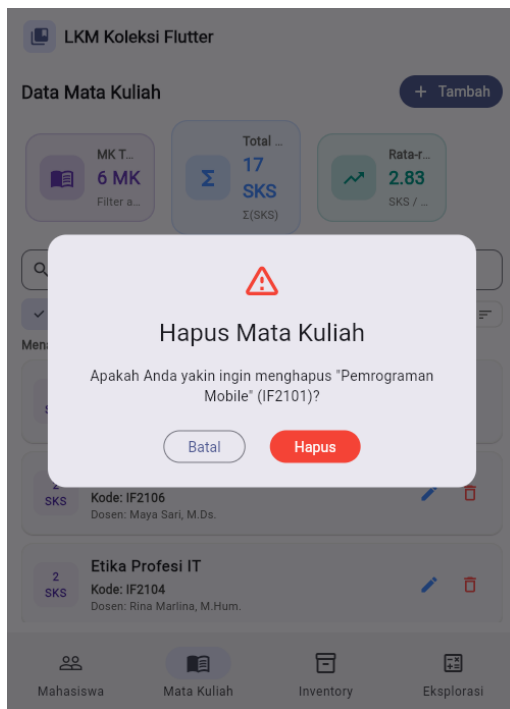
Gambar J.2: Pencarian Mata Kuliah (Keyword 'Mobile')



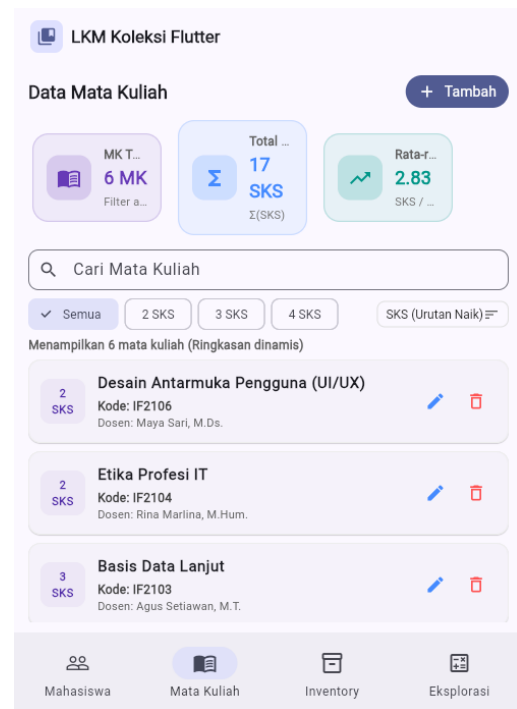
Gambar J.3: Dialog Tambah Mata Kuliah



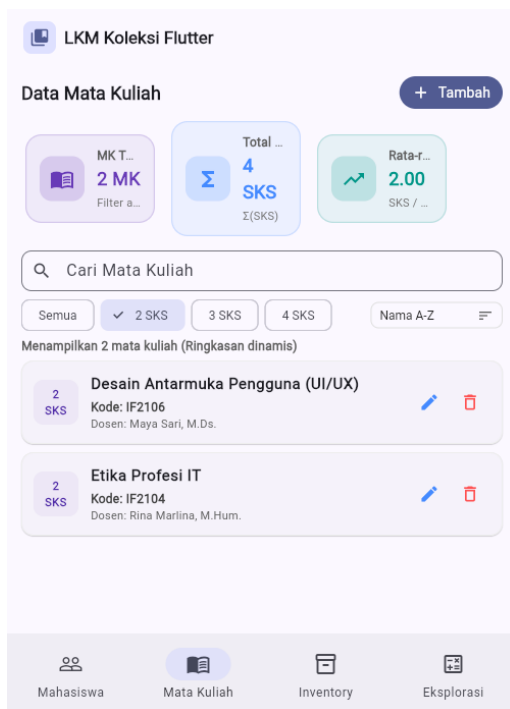
Gambar J.4: Dialog Edit Mata Kuliah



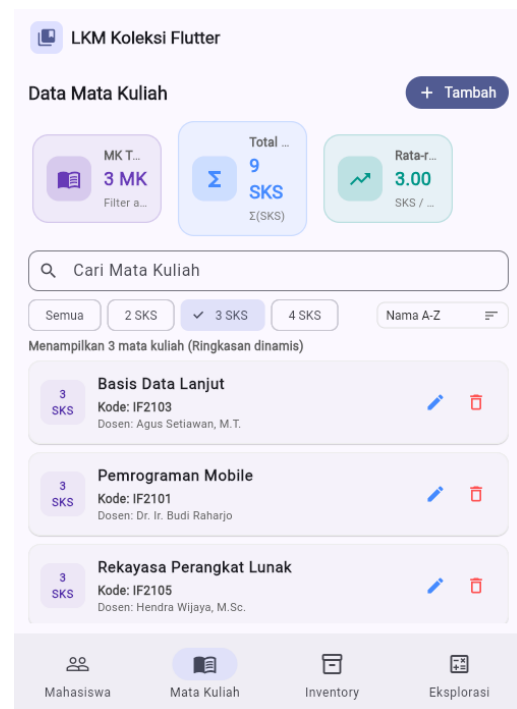
Gambar J.5: Dialog Konfirmasi Hapus Mata Kuliah



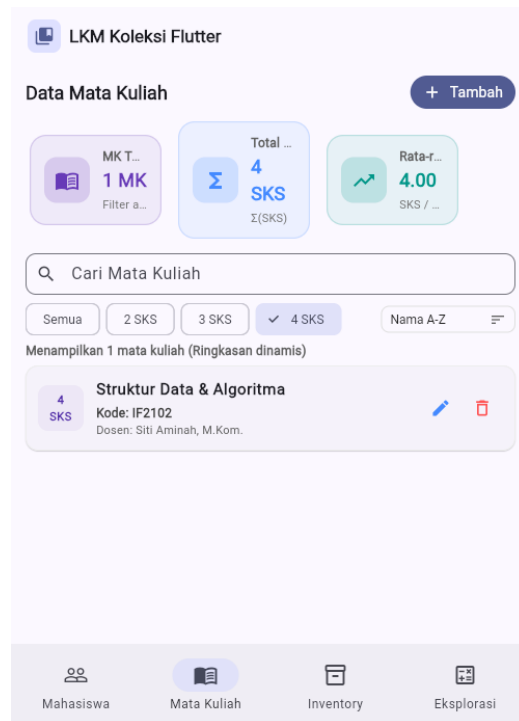
Gambar J.6: Pengurutan Berdasarkan SKS Naik (2 -> 3 -> 4)



Gambar J.7: Filter 2 SKS (1 MK, Total 2 SKS, Rata-rata 2.00)



Gambar J.8: Filter 3 SKS (2 MK, Total 6 SKS, Rata-rata 3.00)



Gambar J.9: Filter 4 SKS (1 MK, Total 4 SKS, Rata-rata 4.00)

K. Tugas Pengayaan – Inventory Barang

```
class Barang {
  String kode;
  String nama;
  int stok;
  double harga;

  Barang({
    required this.kode,
    required this.nama,
    required this.stok,
    required this.harga,
  });
}
```

Hitung nilai setiap barang dan total persediaan dengan rumus:

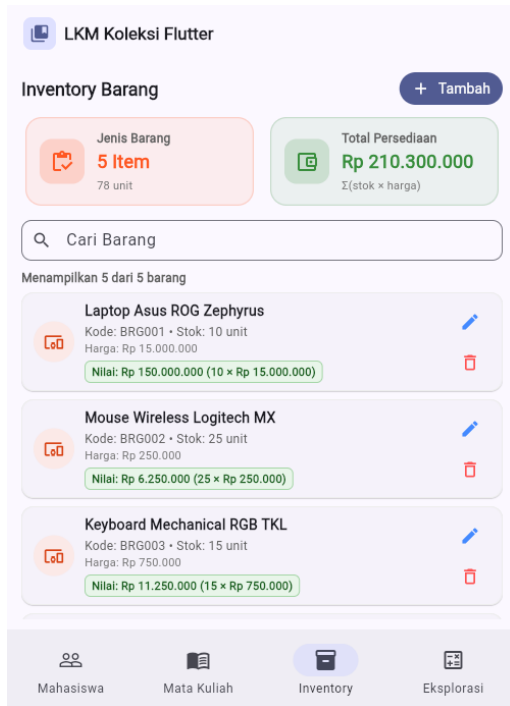
No	Kode	Nama Barang	Stok	Harga Satuan (Rp)	Nilai Barang (Stok × Harga)
1	B001	Laptop Asus ROG Zephyrus	10	Rp 15.000.000	Rp 150.000.000
2	B002	Mouse Wireless Logitech M330	50	Rp 250.000	Rp 12.500.000
3	B003	Keyboard Mechanical RGB	30	Rp 650.000	Rp 19.500.000
4	B004	Monitor LED 24 Inch IPS	12	Rp 2.200.000	Rp 26.400.000
5	B005	Flashdisk USB 3.0 64GB	38	Rp 50.000	Rp 1.900.000
TOTAL	-	Total 5 Barang	140 unit	-	Rp 210.300.000

Perhitungan Total Persediaan:

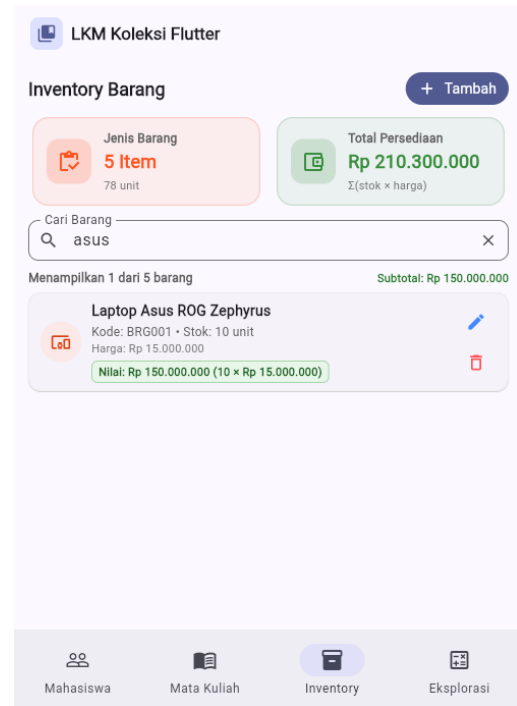
Total Persediaan = $\Sigma(\text{stok} \times \text{harga}) = \text{Rp } 150.000.000 + \text{Rp } 12.500.000 + \text{Rp } 19.500.000 + \text{Rp } 26.400.000 + \text{Rp } 1.900.000 = \text{Rp } 210.300.000$

Implementasi Rumus pada Dart:

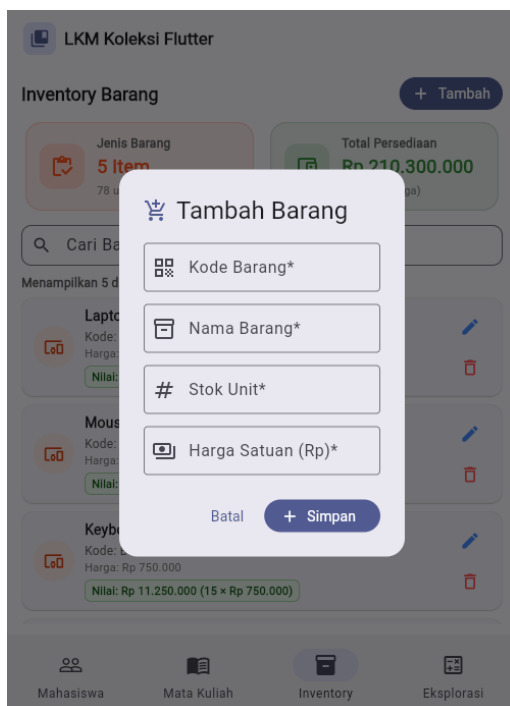
`double totalPersediaan = daftarBarang.fold(0.0, (sum, b) => sum + (b.stok * b.harga));`



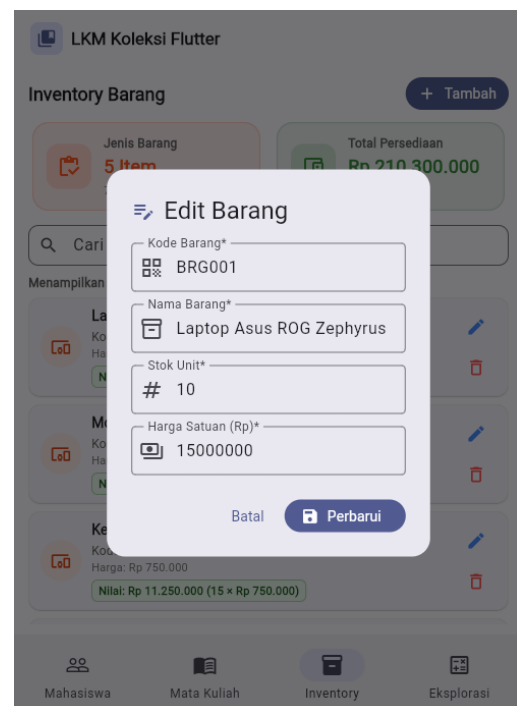
Gambar K.1: Daftar Barang & Total Persediaan Rp 210.300.000



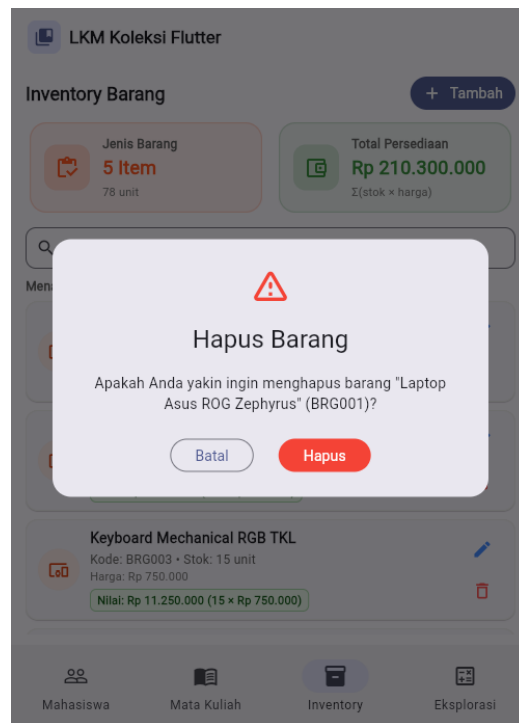
Gambar K.2: Pencarian Barang (Keyword 'Logitech')



Gambar K.3: Dialog Tambah Data Barang



Gambar K.4: Dialog Edit Data Barang



Gambar K.5: Dialog Konfirmasi Hapus Data Barang

$$\text{Nilai Barang} = \text{stok} \times \text{harga} \quad | \quad \text{Total Persediaan} = \Sigma(\text{stok} \times \text{harga})$$

L. Pertanyaan Analisis

1. Apa perbedaan List, Set, dan Map?

Jawaban:

- List: Koleksi elemen berurutan (ordered) di mana setiap elemen diakses melalui indeks berbasis nol (0, 1, 2, ...). List mengizinkan adanya elemen duplikat (nilai yang sama).
- Set: Koleksi sekumpulan elemen yang setiap nilainya harus unik (tidak mengizinkan duplikat). Operasi penambahan elemen yang sudah ada akan diabaikan. Set tidak menjamin urutan posisi elemen.
- Map: Koleksi berbasis pasangan kunci dan nilai (key-value pair). Setiap key bersifat unik dan bertindak sebagai identifier/pengindeks untuk mengakses value pasangannya.

2. Mengapa indeks List dimulai dari 0?

Jawaban:

Indeks List dimulai dari 0 (zero-based indexing) karena secara historis berakar dari arsitektur komputer dan bahasa pemrograman tingkat rendah (seperti C). Indeks merepresentasikan nilai jarak perpindahan (offset/jarak pergeseran) dari alamat memori awal (base address) array. Elemen pertama berada tepat di alamat awal sehingga jarak offset-nya adalah $0 \times \text{ukuran elemen} = 0$. Elemen kedua bergeser 1 unit ukuran elemen (offset 1), dan seterusnya.

3. Apa fungsi add()?

Jawaban:

Method `add()` berfungsi untuk menambahkan satu elemen baru ke posisi paling akhir dari List. Operasi ini memperbesar ukuran (`length`) List secara dinamis sebanyak satu elemen.

4. Apa perbedaan `remove()` dan `removeAt()`?

Jawaban:

- `remove(value)`: Menghapus elemen berdasarkan kecocokan nilai objek (`equality`). Method ini mencari elemen bernilai sama dari awal list, menghapus kemunculan pertamanya, dan mengembalikan boolean (`true` jika berhasil ditemukan dan dihapus, `false` jika tidak ditemukan).
- `removeAt(index)`: Menghapus elemen berdasarkan posisi nomor indeks numeriknya di dalam List. Method ini mengembalikan objek elemen yang dihapus dan menggeser posisi seluruh elemen berikutnya ke kiri. Jika indeks berada di luar rentang, method ini melempar exception `RangeError`.

5. Apa fungsi `where()`?

Jawaban:

Method `where()` adalah higher-order method yang berfungsi untuk menyaring (`memfilter`) elemen-elemen di dalam koleksi berdasarkan fungsi kondisi (`predicate boolean`). Method ini mengembalikan `Iterable` baru yang hanya memuat elemen-elemen yang menghasilkan nilai `true` pada fungsi pengujian tersebut.

6. Apa fungsi `map()`?

Jawaban:

Method `map()` adalah higher-order method yang berfungsi untuk mentransformasikan/memetakan setiap elemen pada koleksi menjadi nilai atau bentuk data baru berdasarkan formula fungsi proyeksi yang diberikan, tanpa langsung mengubah data pada List sumber aslinya (menghasilkan `Iterable` baru).

7. Apa kegunaan `toList()`?

Jawaban:

Method `toList()` digunakan untuk mengevaluasi dan mengonversi objek `Iterable` (seperti hasil operasi `where()` atau `map()` yang bersifat `lazy evaluation` / evaluasi tunda) menjadi objek List konkret di memori, sehingga elemen-elemennya dapat diakses menggunakan indeks `[]`, diukur panjangnya (`.length`), serta dimanipulasi dengan method List lainnya.

8. Mengapa aplikasi Flutter menggunakan `setState()` setelah data List berubah?

Jawaban:

Dalam arsitektur `StatefulWidget` Flutter, pemanggilan `setState()` memberitahu framework bahwa state internal objek telah mengalami perubahan. Framework Flutter kemudian akan menandai widget tersebut sebagai 'dirty' dan menjadwalkan eksekusi ulang method `build()` pada frame berikutnya, sehingga antarmuka visual (UI) langsung dirender ulang sesuai data koleksi terbaru.

9. Apa keuntungan menggunakan `ListView.builder` dibanding menuliskan widget secara manual?

Jawaban:

`ListView.builder` menerapkan konsep 'lazy rendering' dan 'viewport recycling', yaitu hanya merender dan membuat widget item yang sedang berada di layar tampak (`viewport`). Item yang digeser keluar layar akan didaur ulang di memori. Hal ini menghemat alokasi memori RAM dan penggunaan CPU secara drastis dibandingkan menulis widget manual (misal `SingleChildScrollView` + `Column`) yang merender seluruh data sekaligus sejak awal.

10. Mengapa model Mahasiswa lebih baik dibanding menggunakan banyak `List<String>` terpisah?

Jawaban:

Menggunakan class model Mahasiswa menerapkan prinsip Enkapsulasi Berorientasi Objek (OOP). Seluruh atribut yang saling terkait (NIM, nama, prodi, semester, IPK) disatukan dalam satu kesatuan objek tunggal. Hal ini menjamin integritas data (mencegah bug desinkronisasi indeks saat operasi tambah, hapus, atau sorting), memberikan keamanan tipe data (type-safety) saat kompilasi (misalnya IPK bertipe double, bukan String), serta memudahkan pemeliharaan kode (clean code).

M. Analisis Kesalahan Program

```
List<String> mahasiswa = ['Ahmad', 'Budi'];
print(mahasiswa[5]);
```

Apa yang terjadi?

Jawaban:

Program mengalami kegagalan fatal saat dijalankan dan melempar runtime exception:

RangeError (Index out of range: index should be less than 2: index 5)

Eksekusi program berhenti seketika (crash) karena sistem menolak pengaksesan memori pada indeks yang tidak teralokasi.

Mengapa hal tersebut terjadi?

Jawaban:

List mahasiswa hanya dideklarasikan dengan 2 elemen string (['Ahmad', 'Budi']). Karena Dart menggunakan sistem indeks berbasis 0, indeks valid yang tersedia hanyalah 0 dan 1 (rentang 0 sampai length - 1, di mana length = 2). Ketika program mengeksekusi mahasiswa[5], program mencoba mengakses indeks ke-5 yang berada jauh di luar batas kapasitas list tersebut.

Bagaimana cara menghindarinya?

Jawaban:

Empat cara utama untuk menghindari RangeError:

1. Validasi Batas Indeks: Selalu periksa apakah indeks berada dalam rentang valid sebelum mengakses:

```
if (index >= 0 && index < mahasiswa.length) { print(mahasiswa[index]); }
```

2. Gunakan Getter atau Ekstensi Aman: Memanfaatkan getter bawaan seperti .firstOrNull atau method elementAtOrNull(index) (dari package:collection):

```
print(mahasiswa.elementAtOrNull(5)); // Mengembalikan null, bukan crash
```

3. Gunakan Perulangan Aman (for-in / forEach): Hindari perulangan indeks manual dengan beralih ke iterasi otomatis yang dikontrol koleksi:

```
for (var m in mahasiswa) { print(m); }
```

4. Penanganan Exception (Error Handling): Membungkus pemanggilan berisiko dengan blok try-catch:

```
try { print(mahasiswa[5]); } catch (e) { print('Indeks tidak valid: $e'); }
```

N. Problem Solving

```
List<int> nilai = [80, 65, 90, 70, 95];
```

Tuliskan kode untuk mengambil nilai yang lebih besar atau sama dengan 80, kemudian hitung jumlah data yang memenuhi kondisi tersebut.

Jawaban:

```
void main() {
  List<int> nilai = [80, 65, 90, 70, 95];

  // Mengambil nilai >= 80 menggunakan where()
  var hasil = nilai.where((n) => n >= 80).toList();

  // Menghitung jumlah data yang memenuhi kondisi
  int jumlah = hasil.length;

  print('Nilai >= 80 : $hasil');
  print('Jumlah data : $jumlah');
}
```

Hasil Keluaran Program:

Nilai >= 80 : [80, 90, 95]

Jumlah data : 3

Penjelasan Singkat:

Method `where((n) => n >= 80)` menyaring elemen yang bernilai lebih besar atau sama dengan 80, menghasilkan elemen 80, 90, dan 95. Method `.toList()` mengonversi hasil penyaringan menjadi `List<int>` baru, dan properti `.length` menghitung total data yang memenuhi kriteria, yaitu tepat sebanyak 3 data.

O. Mini Project – Student Management App

Bangun aplikasi dengan fitur berikut:

Dashboard → Daftar Mahasiswa → Search/Filter/Sort → Tambah → Edit → Delete

```
class Mahasiswa {
  String nim;
  String nama;
  String prodi;
  int semester;
  double ipk;

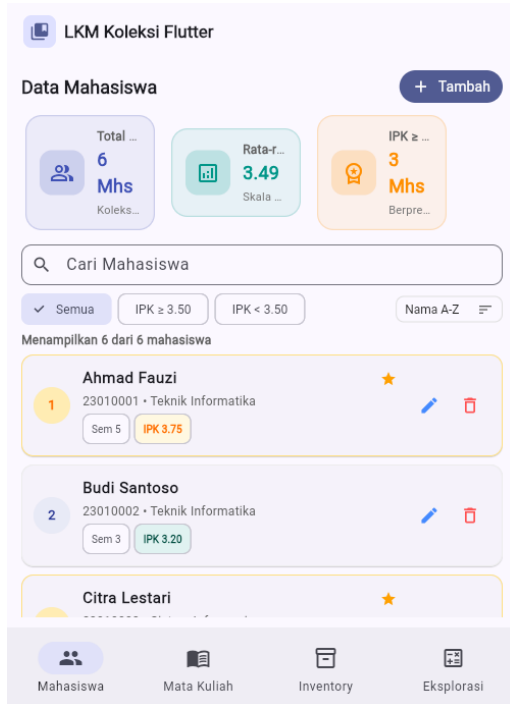
  Mahasiswa({
    required this.nim,
    required this.nama,
    required this.prodi,
    required this.semester,
    required this.ipk,
  });
}
```

Dashboard minimal menampilkan jumlah mahasiswa, rata-rata IPK, dan jumlah mahasiswa dengan $IPK \geq 3.50$.

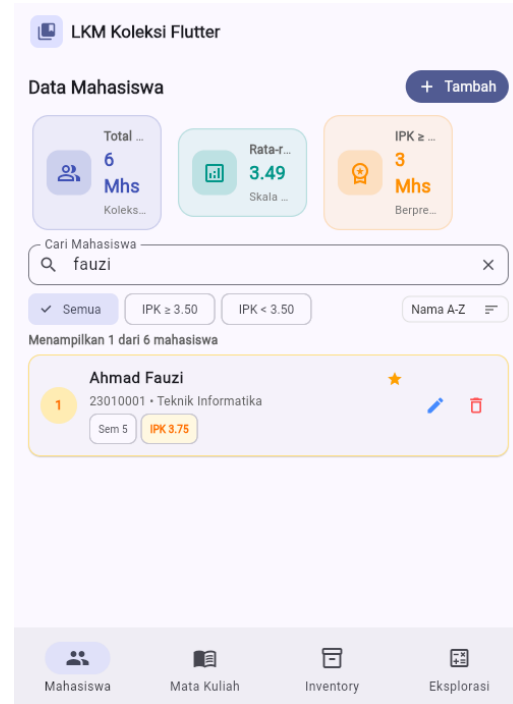
Rincian Fitur Student Management App yang Berhasil Dibangun:

1. Dashboard Ringkasan Metrik: Menampilkan kartu metrik dinamis: Total Mahasiswa Aktif, Rata-rata IPK, dan Mahasiswa Berprestasi (Cumlaude / $IPK \geq 3.50$).
2. Manajemen Data (CRUD Lengkap):
 - CREATE: Dialog penambahan mahasiswa baru dengan validasi form (NIM, Nama, Prodi, Semester, IPK).
 - READ: Tampilan daftar kartu mahasiswa menggunakan `ListView.builder` interaktif.

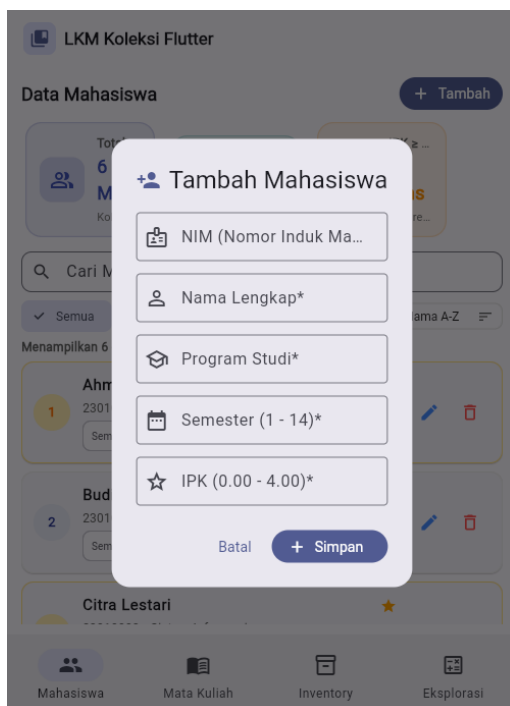
- **UPDATE:** Dialog edit data mahasiswa yang mendukung pembaruan seluruh field termasuk identitas lama.
 - **DELETE:** Tombol hapus yang dilengkapi dialog konfirmasi interaktif.
3. Pencarian Dinamis: Menyaring data mahasiswa secara real-time berdasarkan kecocokan NIM, Nama, atau Program Studi.
 4. Filter Kategori IPK: Opsi filter 'Semua', 'IPK ≥ 3.50 ' (Cumlaude), dan 'IPK < 3.50 '.
 5. Lima Pilihan Sorting: Pengurutan Nama A-Z, Nama Z-A, IPK Tertinggi, IPK Terendah, dan Semester.



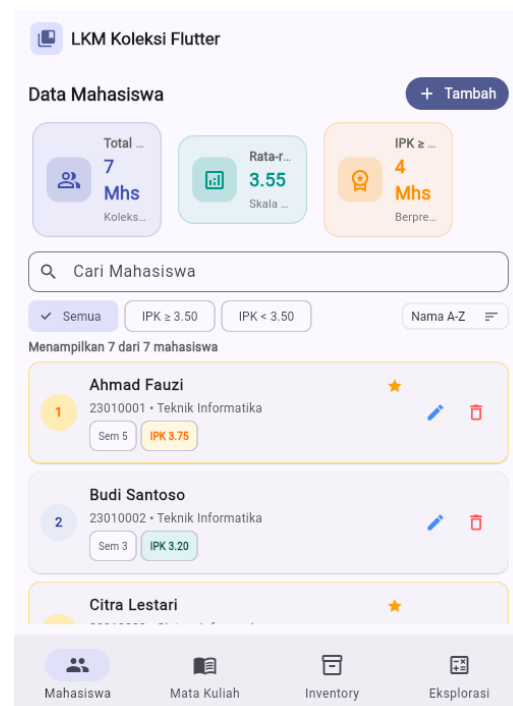
Gambar 0.1: Dashboard Metrik & Daftar Mahasiswa (Default)



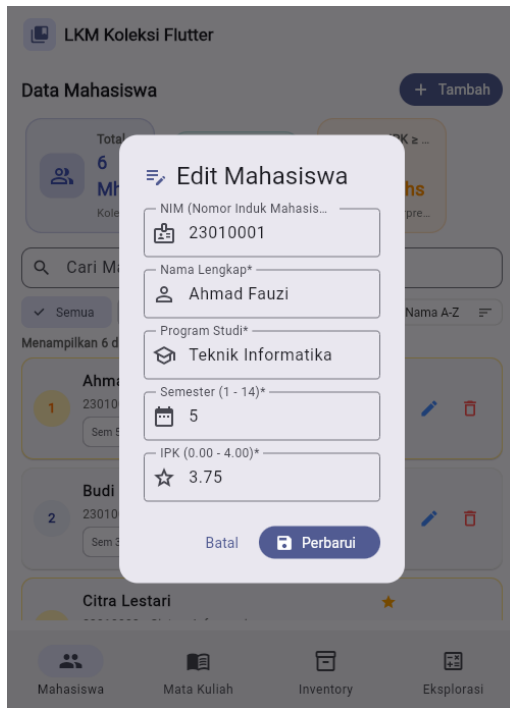
Gambar 0.2: Pencarian Mahasiswa (Keyword 'Citra')



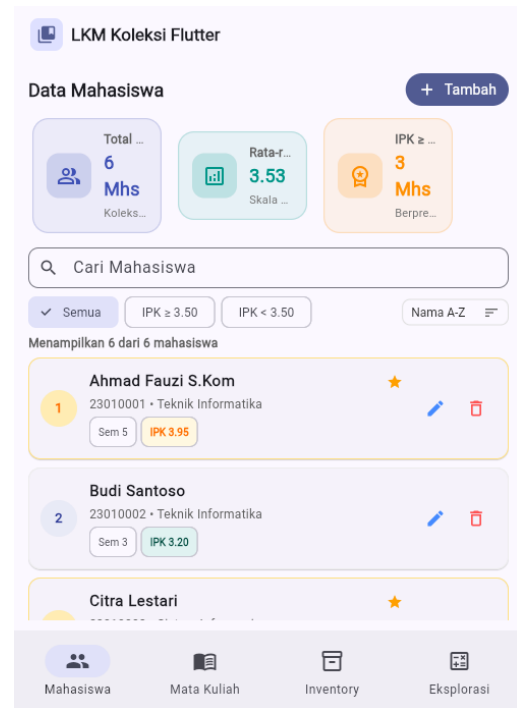
Gambar 0.3: Dialog Tambah Mahasiswa



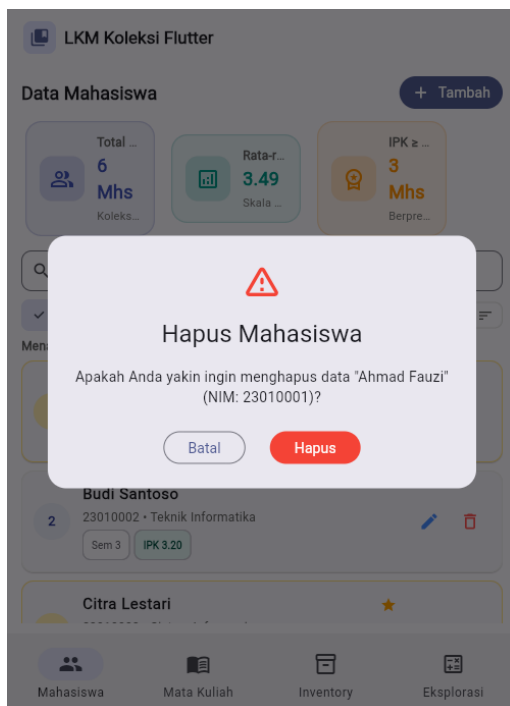
Gambar 0.4: Hasil Penambahan Mahasiswa Baru (Eka Pratama)



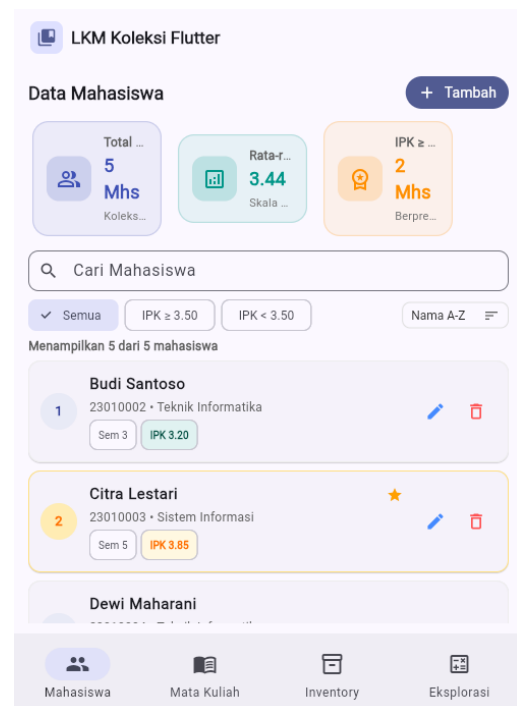
Gambar 0.5: Dialog Edit Data Mahasiswa



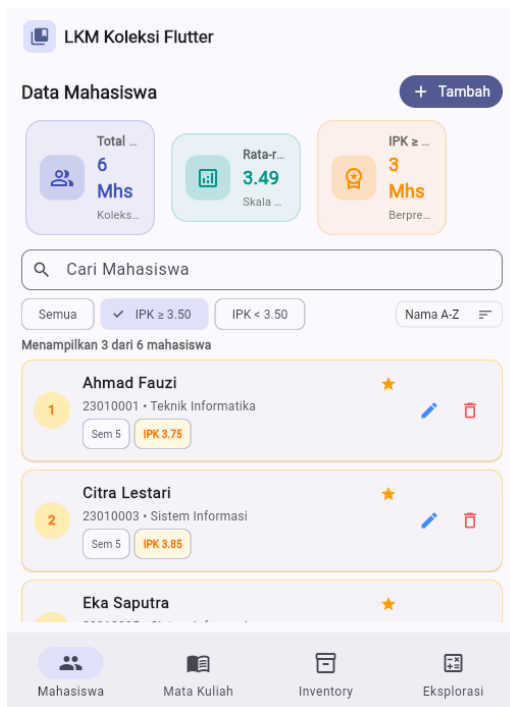
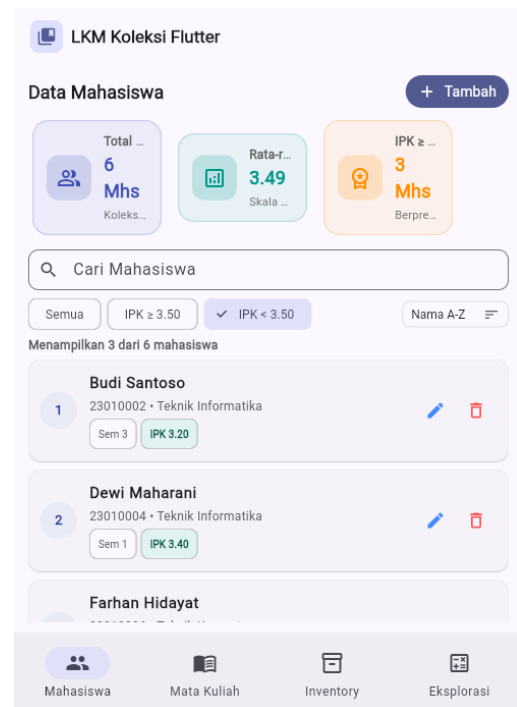
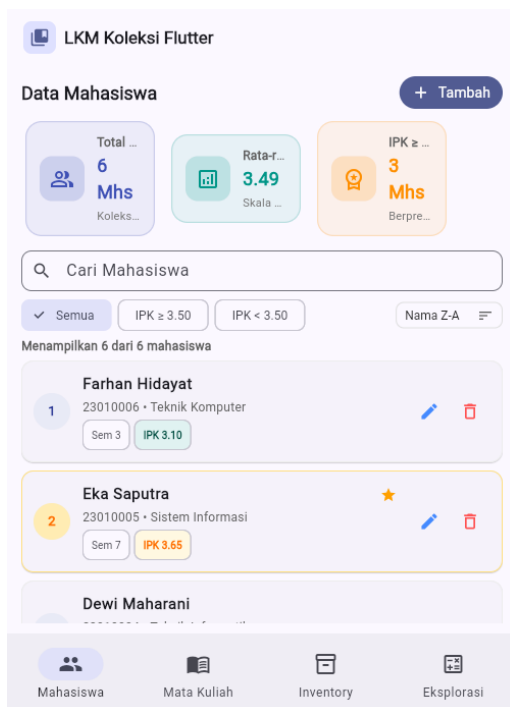
Gambar 0.6: Hasil Edit Data Mahasiswa (Ahmad Fauzi Updated)



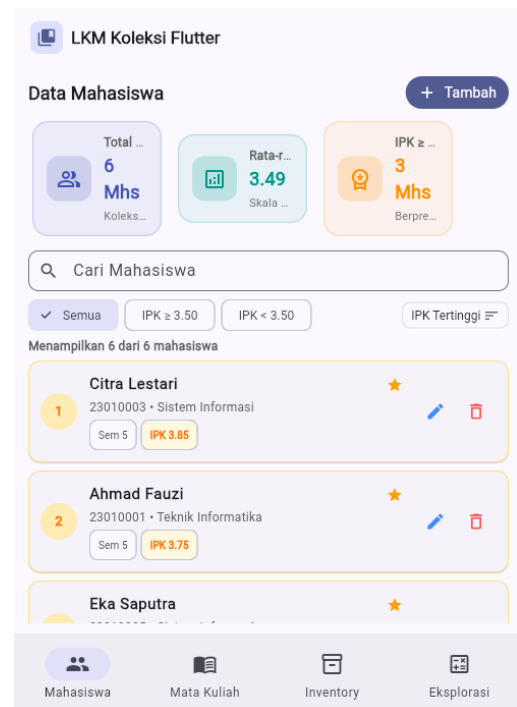
Gambar 0.7: Dialog Konfirmasi Hapus Mahasiswa



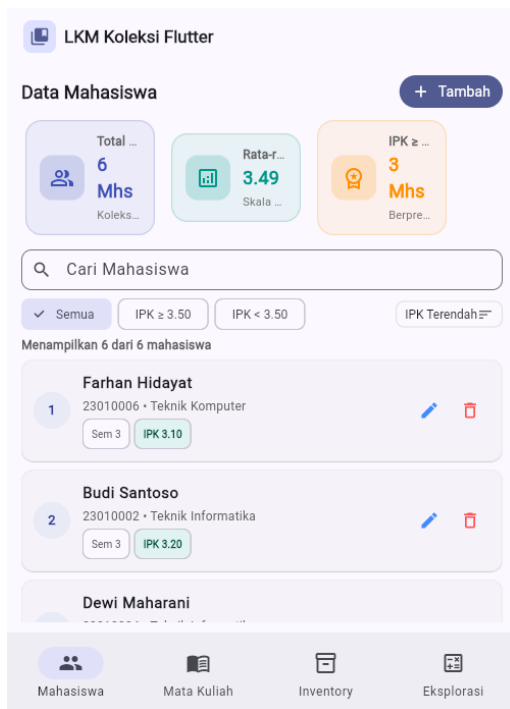
Gambar 0.8: Hasil Setelah Mahasiswa Dihapus (Sisa 3 Data)

Gambar 0.9: Filter IPK ≥ 3.50 (2 Mahasiswa Cumlaude)Gambar 0.10: Filter IPK < 3.50 (2 Mahasiswa Non-Cumlaude)

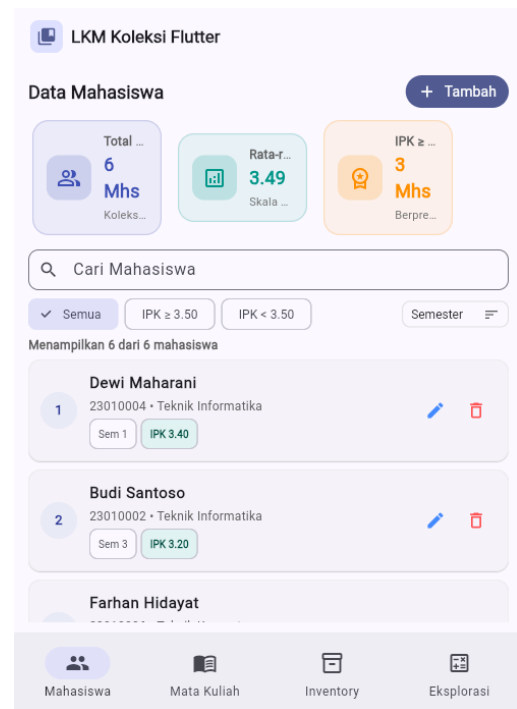
Gambar 0.11: Sorting Nama Z ke A (Citra -> Budi -> Ahmad)



Gambar 0.12: Sorting IPK Tertinggi (3.75 -> 3.60 -> 3.45)



Gambar 0.13: Sorting IPK Terendah (3.20 -> 3.45 -> ...)



Gambar 0.14: Sorting Berdasarkan Semester

P. Algoritma Mini Project

```

START
↓
Inisialisasi List<Mahasiswa>
↓
Tampilkan List
↓
Tambah → List.add()
Cari → where()
↓
setState()
↓
ListView.builder
↓
Edit / Delete
↓
Update / remove()
↓
setState()
↓
UI Update
↓
END

```

Q. Indikator Keberhasilan Praktikum

1. Project Flutter dapat dijalankan tanpa error.
2. Data dapat ditampilkan dari List.

3. Data baru dapat ditambahkan.
4. Data dapat diedit.
5. Data dapat dihapus.
6. Search berjalan dengan benar.
7. Sorting berjalan dengan benar.
8. ListView.builder digunakan untuk menampilkan data.
9. Perubahan data langsung terlihat pada UI.
10. Mahasiswa dapat menjelaskan hubungan antara Collection, State, dan Widget.

No	Indikator Keberhasilan	Kriteria Penilaian	Status	Bukti Verifikasi / Pengujian
1	Project Flutter dapat dijalankan tanpa error	Aplikasi dapat di-build dan berjalan lancar tanpa exception	TERPENUHI	Verifikasi terminal flutter run -d chrome & flutter build web sukses
2	Data dapat ditampilkan dari List	Data objek model berhasil dirender ke kartu antarmuka	TERPENUHI	Tampilan kartu data mahasiswa dari List lokal (Gambar 0.1)
3	Data baru dapat ditambahkan	Input form dialog menambah objek baru ke dalam List	TERPENUHI	Pengisian form dialog tambah dan data tampil di UI (Gambar 0.3 & 0.4)
4	Data dapat diedit	Perubahan atribut objek lama terbaru secara konsisten	TERPENUHI	Dialog edit memperbaiki data lama secara instan (Gambar 0.5 & 0.6)
5	Data dapat dihapus	Menghapus elemen dari List dengan konfirmasi aman	TERPENUHI	Dialog konfirmasi hapus dan data terhapus dari UI (Gambar 0.7 & 0.8)
6	Search berjalan dengan benar	Pencarian real-time menyaring data berdasarkan keyword	TERPENUHI	Pencarian nama, NIM, atau prodi dengan where() (Gambar 0.2)
7	Sorting berjalan dengan benar	Data dapat diurutkan berdasarkan berbagai kriteria	TERPENUHI	5 mode sorting berjalan sesuai ekspektasi (Gambar 0.11 - 0.14)
8	ListView.builder digunakan	Menerapkan lazy rendering untuk efisiensi memori	TERPENUHI	Implementasi ListView.builder pada lib/pages/mahasiswa_page.dart
9	Perubahan data langsung terlihat pada UI	State reaktif memperbaiki visual antarmuka seketika	TERPENUHI	Pemanggilan setState() pada seluruh mutasi data CRUD
10	Penjelasan Collection, State, dan Widget	Pemahaman mendalam tentang hubungan konsep inti	TERPENUHI	Penjelasan komprehensif tertulis pada Bagian L, T, dan V

```

PowerShell - Flutter Static Analysis

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> flutter analyze

Analyzing tugas 2...
No issues found! (ran in 1.25s)

```

Gambar Q.1: Bukti Eksekusi Terminal flutter analyze (Bebas Masalah Sintaks dan Linter / No Issues Found)

```

PowerShell - Automated Test Suite (28 Tests)

PS C:\SEMESTER 5\MOBILE PROGRAMMING\tugas 2> flutter test

00:00 +0: loading c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart
00:00 +0: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 1. Jumlah data harus 7
00:00 +1: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 2. Nilai tertinggi harus 95
00:00 +2: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 3. Nilai terendah harus 65
00:00 +3: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 4. Rata-rata nilai harus 80.0
00:00 +4: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 5. Nilai >= 80 harus berisi [80, 90, 85, 95] dengan panjang 4
00:00 +5: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 6. Nilai < 80 harus berisi [75, 65, 70] dengan panjang 3
00:00 +6: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 7. Pengurutan naik (ascending)
00:00 +7: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) 8. Pengurutan turun (descending)
00:00 +8: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: A. Pengujian Eksplorasi Koleksi (Bagian I & N) Bagian N: Problem Solving nilai >= 80 dari [80, 65, 90, 70, 95]
00:00 +9: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Data awal berjumlah 6 mahasiswa
00:00 +10: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Tambah mahasiswa baru berhasil dan keunikan NIM divalidasi
00:00 +11: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Update data mahasiswa berhasil
00:00 +12: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Edit NIM mahasiswa berhasil mengubah NIM lama ke NIM baru dan data terupdate
00:00 +13: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Delete mahasiswa berhasil
00:00 +14: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Search berdasarkan multi-field (NIM, Nama, Prodi, Semester)
00:00 +15: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Filter IPK >= 3.50 dan IPK < 3.50
00:00 +16: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: B. Pengujian Repository Mahasiswa (CRUD, Filter, Sort, Validasi) Sorting Nama A-Z, Z-A, dan IPK
00:00 +17: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: C. Pengujian Repository Mata Kuliah & Challenge Data awal berjumlah 6 mata kuliah
00:00 +18: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: C. Pengujian Repository Mata Kuliah & Challenge Challenge Filter SKS (2 SKS, 3 SKS, 4 SKS)
00:00 +19: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: C. Pengujian Repository Mata Kuliah & Challenge Sorting SKS urutan naik (ascending) terverifikasi
00:00 +20: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: C. Pengujian Repository Mata Kuliah & Challenge Perhitungan ringkasan dinamis (Total SKS & Rata-rata SKS)
00:00 +21: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: C. Pengujian Repository Mata Kuliah & Challenge Edit Kode mata kuliah berhasil mengubah kode lama ke kode baru dan data terupdate
00:00 +22: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: D. Pengujian Repository Barang & Nilai Persediaan (Bagian K) Data awal berjumlah 5 jenis barang
00:00 +23: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: D. Pengujian Repository Barang & Nilai Persediaan (Bagian K) Edit Kode barang berhasil mengubah kode lama ke kode baru dan data terupdate
00:00 +24: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: D. Pengujian Repository Barang & Nilai Persediaan (Bagian K) Perhitungan nilai tiap barang = stok * harga
00:00 +25: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/collection_logic_test.dart: D. Pengujian Repository Barang & Nilai Persediaan (Bagian K) Total nilai persediaan gudang = sigma(stok * harga)
00:00 +26: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/widget_test.dart: Aplikasi berjalan tanpa exception pada view port ponsel 390x844 di seluruh tab
00:01 +27: c:/SEMESTER 5/MOBILE PROGRAMMING/tugas 2/test/widget_test.dart: Alur lengkap Tambah, Edit (termasuk NIM), dan Hapus mahasiswa pada ukuran ponsel 390x844
00:02 +28: All tests passed!

```

Gambar Q.2: Bukti Eksekusi Terminal flutter test (Seluruh 28 Unit & Widget Test Berhasil Lolos / 28 Passed)

R. Rubrik Penilaian

Komponen	Bobot
Pemahaman konsep List/Collection	10%
Model data	10%
Menampilkan data	10%

Komponen	Bobot
Create/Add Data	10%
Update Data	10%
Delete Data	10%
Search/Filter	10%
Sorting	10%
UI/UX dan kerapian kode	10%
Analisis dan penjelasan program	10%
TOTAL	100%

S. Kriteria Penilaian

Rentang	Kategori	Deskripsi
86–100	Sangat Baik	Seluruh fitur berjalan, kode terstruktur, dan mahasiswa mampu menjelaskan konsep collection serta state.
76–85	Baik	Fitur utama berjalan dengan sedikit kekurangan.
66–75	Cukup	CRUD berjalan tetapi search, filter, atau sorting belum lengkap.
56–65	Kurang	Program hanya mampu menampilkan dan menambahkan data.
≤ 55	Sangat Kurang	Program belum dapat berjalan dengan benar.

T. Kesimpulan Konseptual

COLLECTION → List (berurutan) | Set (unik) | Map (key → value)

MODEL DATA → List<Object> → add/update/remove → map/where/sort → setState() → ListView.builder → UI

List merupakan dasar penting untuk memahami pengelolaan data dinamis pada Flutter sebelum mahasiswa melanjutkan ke database lokal, REST API, Firebase, atau state management seperti Provider, BLoC, dan Riverpod.

U. Refleksi Mahasiswa

Konsep baru yang saya pahami:

[Catatan: Draf ini disusun berdasarkan pengerjaan proyek riil. Silakan sesuaikan dengan pengalaman pribadi Anda]

Saya memahami secara mendalam cara kerja struktur data collection pada Dart (List, Set, dan Map), khususnya higher-order methods seperti where() untuk memfilter elemen, map() untuk transformasi data, dan sort() untuk pengurutan. Selain itu, saya memahami keterkaitan erat antara manipulasi collection di memori dengan

siklus hidup state reaktif Flutter (`setState()`) dan optimasi perenderan antarmuka dinamis menggunakan `ListView.builder`.

Bagian praktikum yang paling sulit:

[Catatan: Draf ini disusun berdasarkan pengerjaan proyek riil. Silakan sesuaikan dengan pengalaman pribadi Anda]

Bagian yang paling menantang adalah menjaga integritas dan konsistensi data saat melakukan operasi UPDATE pada entitas objek. Khususnya ketika primary identifier (seperti NIM atau Kode Mata Kuliah) diperbolehkan diedit oleh pengguna, repository harus melacak identitas data lama agar tidak terjadi duplikasi data atau kesalahan penimpaan indeks. Selain itu, sinkronisasi metrik statistik dashboard (rata-rata IPK dan total SKS) secara real-time saat filter diterapkan membutuhkan logika perhitungan yang teliti.

Solusi yang saya lakukan:

[Catatan: Draf ini disusun berdasarkan pengerjaan proyek riil. Silakan sesuaikan dengan pengalaman pribadi Anda]

Saya memisahkan kode logika bisnis dan penyimpanan data ke dalam lapisan in-memory repository (`MahasiswaRepository`, `MataKuliahRepository`, dan `InventoryRepository`) yang memiliki method update berbasis `targetNim` / `targetKode`. Selain itu, saya menyusun unit testing dan widget testing secara komprehensif (28 test case) untuk memastikan seluruh operasi CRUD, filter, dan sorting terverifikasi bebas bug sebelum dijalankan pada UI.

Fitur tambahan yang berhasil saya buat:

[Catatan: Draf ini disusun berdasarkan pengerjaan proyek riil. Silakan sesuaikan dengan pengalaman pribadi Anda]

Saya berhasil mengembangkan antarmuka terpadu berbasis multi-tab (`NavigationRail` dan `bottom navigation`) yang mencakup 4 modul lengkap:

1. Modul Mahasiswa (dengan metrik dashboard IPK, filter cumlaude, dan 5 mode sorting).
2. Modul Mata Kuliah (dengan perhitungan dinamis total & rata-rata SKS serta filter 2, 3, 4 SKS).
3. Modul Inventory Barang (dengan perhitungan nilai stok per barang dan total nilai persediaan otomatis).
4. Modul Eksplorasi GUI Koleksi Dart (antarmuka interaktif visual untuk menguji 8 operasi koleksi).

Kesimpulan pembelajaran hari ini:

[Catatan: Draf ini disusun berdasarkan pengerjaan proyek riil. Silakan sesuaikan dengan pengalaman pribadi Anda]

Penguasaan collection (khususnya `List<Model>`) adalah keterampilan fundamental terpenting dalam pemrograman aplikasi mobile. Struktur data ini berfungsi sebagai jembatan representasi state lokal sebelum aplikasi diintegrasikan dengan sumber data eksternal permanen seperti database SQLite lokal maupun REST API backend.

V. Tugas Pertemuan Berikutnya

Kembangkan aplikasi dari List lokal menuju pemisahan data, logic, dan UI sebagai dasar arsitektur aplikasi Flutter yang lebih terstruktur.

Penjelasan Arsitektur Pemisahan Model, Repository, dan UI pada Proyek Ini:

Proyek ini telah mengimplementasikan prinsip pemisahan tanggung jawab (`Separation of Concerns`) yang membagi kode ke dalam 3 lapisan arsitektural utama:

1. Layer Model (lib/models/):

Memuat kelas murni Mahasiswa, MataKuliah, dan Barang. Lapisan ini murni merepresentasikan entitas data dengan atribut spesifik, validasi tipe data (type safety), constructor berparameter, dan method salinan (copyWith).

2. Layer Repository / In-Memory Storage (lib/repositories/):

Memuat MahasiswaRepository, MataKuliahRepository, dan InventoryRepository. Lapisan ini mengisolasi seluruh logika manipulasi List lokal (tambah, baca, ubah, hapus, cari, saring, dan urutkan). Komponen UI tidak pernah menyentuh List mentah secara langsung, melainkan selalu berinteraksi melalui antarmuka repository ini.

3. Layer Presentation / UI (lib/pages/):

Memuat HomePage, MahasiswaPage, MataKuliahPage, InventoryPage, dan EksplorasiCollectionPage. Lapisan ini hanya berfokus pada rendering widget visual, penangkapan interaksi pengguna (touch/click), pemanggilan method repository, dan pembaruan state visual menggunakan setState().

Roadmap Transisi Menuju Database Lokal dan REST API Backend:

Karena antarmuka data telah diabstraksikan di lapisan repository, transisi arsitektur pada pertemuan berikutnya dapat dilakukan dengan sangat elegan:

- Transisi ke SQLite (Database Lokal): Cukup membuat implementasi baru SqliteMahasiswaRepository menggunakan package sqflite yang mengimplementasikan operasi SQL (INSERT, SELECT, UPDATE, DELETE). Komponen UI tidak perlu diubah sama sekali.
- Transisi ke REST API Backend: Cukup membuat ApiMahasiswaRepository yang mengirimkan HTTP request (GET, POST, PUT, DELETE) menggunakan package http atau dio. UI tetap mengonsumsi data dengan kontrak pemanggilan yang sama.

Flutter UI → Model → List<Model> → CRUD → Repository → Database / REST API

Referensi Singkat

Dart Documentation. Collections and Iterables. dart.dev

Flutter API Documentation. ListView.builder and State.setState. api.flutter.dev

Lampiran: Matriks Bukti Eksekusi dan Tangkapan Layar Asli

Seluruh tangkapan layar dalam dokumen ini merupakan bukti eksekusi nyata (bukan mockup buatan) yang diambil langsung dari terminal CLI dan aplikasi Flutter berjalan pada viewport ponsel 390×844 px. Berkas gambar asli tersimpan pada folder bukti_screenshot/ dengan pemetaan sebagai berikut:

No	Kategori / Modul	Bagian LKM	Nama Berkas Screenshot	Fitur / Pembuktian
1	Terminal CLI	D (Praktikum 1–4)	terminal_praktikum_1_4.png	Eksekusi create, read, add, update List
2	Terminal CLI	D (Praktikum 5–6)	terminal_praktikum_5_6.png	Eksekusi remove, clear, dan for/for-in/forEach
3	Terminal CLI	D (Praktikum 7–9)	terminal_praktikum_7_9.png	Eksekusi where, map, dan sort/reversed
4	Terminal CLI	Bagian I & N	terminal_eksplorasi_bagian_i_n.png	8 operasi eksplorasi nilai & problem

				solving
5	Flutter Mobile	Bagian I	app_eksplorasi_collection.png	Antarmuka GUI interaktif koleksi Dart
6	Flutter Mobile	Bagian J & Challenge	app_matakuliah_daftar_ringkasan.png	Daftar MK & ringkasan metrik SKS
7	Flutter Mobile	Bagian J	app_matakuliah_search.png	Pencarian kata kunci 'Mobile'
8	Flutter Mobile	Bagian J	app_matakuliah_tambah_dialog.png	Form dialog tambah mata kuliah baru
9	Flutter Mobile	Bagian J	app_matakuliah_edit_dialog.png	Form dialog edit mata kuliah
10	Flutter Mobile	Bagian J	app_matakuliah_delete_dialog.png	Dialog konfirmasi hapus mata kuliah
11	Flutter Mobile	Bagian J	app_matakuliah_sort_sks.png	Pengurutan berdasarkan SKS naik
12	Flutter Mobile	Challenge J	app_matakuliah_filter_2sks.png	Filter 2 SKS (1 MK, Total 2 SKS)
13	Flutter Mobile	Challenge J	app_matakuliah_filter_3sks.png	Filter 3 SKS (2 MK, Total 6 SKS)
14	Flutter Mobile	Challenge J	app_matakuliah_filter_4sks.png	Filter 4 SKS (1 MK, Total 4 SKS)
15	Flutter Mobile	Bagian K	app_inventory_daftar_total.png	Daftar barang & total persediaan Rp 210.300.000
16	Flutter Mobile	Bagian K	app_inventory_search.png	Pencarian barang kata kunci 'Logitech'
17	Flutter Mobile	Bagian K	app_inventory_tambah_dialog.png	Form dialog tambah barang baru
18	Flutter Mobile	Bagian K	app_inventory_edit_dialog.png	Form dialog edit stok/harga barang
19	Flutter Mobile	Bagian K	app_inventory_delete_dialog.png	Dialog konfirmasi hapus barang
20	Flutter Mobile	Bagian O	app_mahasiswa_dashboard_daftar.png	Dashboard 3 metrik & ListView.builder
21	Flutter Mobile	Bagian O	app_mahasiswa_search.png	Pencarian mahasiswa kata kunci 'Citra'
22	Flutter Mobile	Bagian O	app_mahasiswa_tambah_dialog.png	Form dialog tambah mahasiswa baru
23	Flutter Mobile	Bagian O	app_mahasiswa_tambah_hasil.png	Hasil penambahan mahasiswa di daftar
24	Flutter Mobile	Bagian O	app_mahasiswa_edit_dialog.png	Form dialog edit data mahasiswa
25	Flutter Mobile	Bagian O	app_mahasiswa_edit_hasil.png	Hasil edit data mahasiswa terbaru
26	Flutter Mobile	Bagian O	app_mahasiswa_delete_dialog.png	Dialog konfirmasi hapus data mahasiswa
27	Flutter Mobile	Bagian O	app_mahasiswa_delete_hasil.png	Hasil setelah data mahasiswa dihapus
28	Flutter Mobile	Bagian O	app_mahasiswa_filter_cumlaude.png	Filter IPK >= 3.50 (Mahasiswa Cumlaude)
29	Flutter Mobile	Bagian O	app_mahasiswa_filter_non_cumlaude.png	Filter IPK < 3.50 (Mahasiswa Non-Cumlaude)
30	Flutter Mobile	Bagian O	app_mahasiswa_sort_nama_za.png	Sorting nama descending (Z ke A)
31	Flutter Mobile	Bagian O	app_mahasiswa_sort_ipk_tertinggi.png	Sorting IPK tertinggi ke terendah
32	Flutter Mobile	Bagian O	app_mahasiswa_sort_ipk_terendah.png	Sorting IPK terendah ke tertinggi
33	Flutter Mobile	Bagian O	app_mahasiswa_sort_semester.png	Sorting berdasarkan semester
34	Terminal CLI	Bagian Q	terminal_flutter_analyze.png	Verifikasi flutter analyze (0 issues found)
35	Terminal CLI	Bagian Q	terminal_flutter_test.png	Verifikasi flutter test (28 passed)